Week 11 — Threads and Concurrency

Format: seven hands-on-first lessons using the canonical `projects/linux-system-programming` lab

Suggested workload: approximately 21 focused hours, adjustable to mastery

> Predict an invariant, run a bounded experiment, identify the happens-before edges and object lifetimes, and stop the claim where the evidence stops.

The project is an offline C17/POSIX threads lab. This week does not copy or patch it. Every wrapper resolves the canonical project, builds through its existing Makefile, labels expected failures and optional-tool skips, and uses:

```
purpose → prediction → evidence → explanation → limits
```

Run wrappers from this week directory. Output ordering, thread IDs, timings, and relaxed-memory observations are intentionally not treated as stable.

Daily roadmap

1. Threads, processes, stacks, and TLS — pthread/process relationships, clone intuition, shared address spaces, per-thread stacks, TLS, joins, and lifetime.
2. Data races and the C abstract machine — conflicting actions, undefined behavior, failed increments, compiler freedom, and why plausible output proves nothing.
3. Mutexes, ownership, ordering, and futexes — protected invariants, ownership transfer, global lock order, and the userspace/kernel boundary.
4. Condition variables and bounded queues — predicates, while loops, spurious wakeups, lost-wakeup avoidance, and a producer-consumer proof.
5. C11 atomics and memory orders — modification order, release/acquire, seq_cst, compiler versus CPU ordering, and cautious x86-64/AArch64/RISC-V mappings.
6. Liveness, false sharing, and measurement — deadlock, starvation, cache-line interference, scalability, and what a microbenchmark cannot establish.
7. Integrated debugging report — GDB, Valgrind, sanitizers, intentional failures, evidence quality, and ToyOS/xv6/Linux comparison.

Each day has one executable wrapper and a generated four-question HTML theory check. `quizzes.json` is their source. The consolidated `week-11-material.pdf` preserves the overview and all seven lessons for offline reading.

Safety and evidence discipline

The intentional data race has undefined behavior and is excluded from the ordinary deterministic suite. Run it only through Day 2 or Day 7's bounded wrapper. A ThreadSanitizer build does not imply its runtime can execute on this host: compile failure, runtime incompatibility, and a completed run that did not report the race are reported separately. Absence of a report is not proof of absence.

The deadlock exercise uses timed locking and terminates. It demonstrates a wait-for cycle without leaving a hung process. The false-sharing and relaxed litmus runs are measurements, not correctness tests. In particular, an x86-64 observation cannot prove portable C behavior; the C abstract machine defines portability, while ISA manuals and compiler output explain one implementation.

Completion checkpoint

Your report should answer:

1. Which state is process-wide, which is per-thread, and who owns each object?
2. Which operations conflict, and what happens-before edge—if any—orders them?
3. Why can `counter++` fail at both the machine and language levels?
4. What invariant does each mutex protect, and what global lock order holds?
5. Why must condition waits re-check a predicate while holding its mutex?
6. Which atomic ordering is sufficient for the stated protocol, and why?
7. Which claims came from program assertions, timing, debugger, dynamic tool, compiler output, ISA documentation, or source?
8. What remains unproved because of scheduling, optimization, architecture, tool availability, runtime policy, sampling, or workload choice?

Do not auto-fill mastery, confidence, hours, or completion.

Day 1 — Threads, Processes, `clone`, Stacks, and TLS

Target time: approximately 3 hours

> A thread is not “a small process.” Start from which resources are shared, which execution state is private, and which lifetime event makes access safe.

1. Run before naming the machinery

Run `challenges/day-001-thread-model.sh`. Before running it, predict the one stable line from `thread_lab basic`: creation succeeds, the worker returns the same pointer it received, and `pthread_join` gives that pointer back to the creator. Predict nothing about numeric thread IDs or scheduling order.

The mode passes the address of `token`, an automatic object in `mode_basic`, to the worker. This is safe only because the creating thread remains inside that function until after the join. The join is both a lifetime decision and a synchronization event. If the function returned, or if the object belonged to a detached thread's expired caller frame, dereferencing the pointer would be invalid even if the bytes appeared unchanged.

Record the binary hash, compiler version, architecture, and kernel release. These identify the artifact and environment. They do not turn a single run into a universal scheduling claim.

2. Relate a pthread to a process precisely

POSIX describes threads within a process. Threads share the process address space and many process resources: mapped code and data, heap objects, open file descriptions reached through descriptors, signal dispositions, current working directory, and credentials. Each thread has its own execution context: registers, stack, scheduling state, blocked signal mask, `errno`, and thread-specific data.

“The heap is shared” is shorthand, not a safety property. Sharing means each thread can address the same allocated object. It does not establish ownership, object lifetime, synchronization, or permission to perform conflicting accesses. A pointer being reachable by two threads is exactly where a design must state who may read, mutate, and free it.

A process can have one thread or many. Creating a pthread does not create an independent POSIX process with a new address space. Conversely, Linux exposes tasks as a flexible implementation concept; resource-sharing choices make tasks behave as threads or process-like children. Keep the POSIX contract separate from Linux's internal representation.

3. Build `clone` intuition without replacing the API

On Linux, libc commonly creates pthreads using a `clone`-family system call with flags that request sharing of the virtual memory, file-descriptor table, signal-handler table, and thread-group identity, plus thread-lifecycle coordination. Exact flags, syscall spelling (`clone` or `clone3`), setup calls, and TLS details vary by libc, kernel, architecture, and version.

This is useful intuition: Linux task creation is parameterized resource sharing, not two unrelated primitives implemented from nothing. It is not an application contract. Correct C should use `pthread_create`, `pthread_join`, and documented synchronization rather than calling raw `clone` based on one trace. A trace also shows a decoded syscall boundary, not every libc action or kernel-internal branch.

Compare with `fork`: after `fork` in a multithreaded process, only the calling thread exists in the child, and until `exec` the child is severely constrained in what it may safely call. A pthread creation instead adds concurrent execution inside the shared process. Copy-on-write after `fork` and shared address space between pthreads are different semantics.

4. Understand the per-thread stack

Every thread needs stack storage for call frames, automatic objects, saved registers, and implementation bookkeeping. Stacks occupy distinct virtual address ranges in the same address space. “Private stack” means the runtime assigns it to one thread's execution; it does not mean hardware makes its addresses inaccessible to peers. If one thread publishes a pointer into its stack, another can often dereference it while the object remains alive.

That capability is dangerous. The pointed-to automatic object's lifetime ends when its block exits. The entire thread stack ceases to support its old frames when the thread terminates. `pthread_join` waits for termination, but joining does not revive expired automatic objects. A worker must not return a pointer to its own local variable. Safe patterns include caller-owned storage whose lifetime spans the join, heap allocation with an explicit transfer protocol, or copying a result into synchronized shared storage.

Stack size is finite and configurable. Deep recursion or large automatic arrays can exhaust a worker stack. Measuring the main thread's stack limit does not necessarily identify every pthread's configured size. Guard pages and allocation policy are implementation details worth observing but not assuming.

5. Distinguish TLS from ordinary globals

Thread-local storage gives each thread a distinct instance of an object declared with C's `_Thread_local` or provided through pthread-specific keys. The declaration can look global while the storage is per-thread. Addresses, initialization, dynamic-loader support, and access sequences depend on the ABI and model selected by the toolchain.

TLS solves “one instance per thread”; it does not solve arbitrary cross-thread communication. Publishing a pointer to one thread's TLS reintroduces lifetime and synchronization questions. Destructors for pthread keys run under defined termination rules, but code must tolerate iteration limits and destructor ordering constraints. `errno` is the familiar conceptual example: one thread's failed operation must not overwrite another's diagnostic state.

Contrast four locations:

- a global counter is one shared object and needs synchronization for writes;
- a heap allocation is shared if its pointer is shared, with ownership chosen by the program;
- a stack local is tied to block/thread lifetime even if its address escapes;
- a TLS object has a distinct instance and thread-related lifetime.

Storage location alone never supplies a complete ownership policy.

6. Treat creation and join as synchronization

Actions sequenced before a successful `pthread_create` are visible to the new thread under the pthread synchronization contract. Termination and successful join establish the corresponding completion relationship: the joiner can consume results produced by the finished thread. This is why the basic mode does not need a mutex around `token`: the creator initializes it before creation, the worker changes no shared state, and the creator checks only after join.

If both threads modified `token` concurrently, creation and join would not order the middle of their executions. “I eventually join” does not legalize a race that occurred before the join. Synchronization must order every conflicting non-atomic access.

Join also reclaims thread resources and documents one owner of the returned value. A joinable thread should be joined exactly once. A detached thread cannot later be joined, so any data it uses must outlive its asynchronous work through another protocol. `pthread_t` is an opaque handle; do not print or compare it as though it were a portable integer, and do not confuse it with a Linux TID.

7. Debug lifetime before timing

When a threaded program intermittently fails, first draw an object-lifetime table: allocation, publication, permitted users, synchronization, final use, and destruction. Then draw the happens-before graph. Only after both are coherent should scheduling or performance become the primary hypothesis.

Typical failures include passing the address of a loop variable to several workers, freeing a task immediately after enqueueing it, returning a stack address, destroying a mutex while a waiter may still reference it, or letting `main` return while detached workers retain pointers into process state. These are C lifetime errors or protocol errors, not mysterious pthread behavior.

Practical challenge

Purpose: connect pthread creation and joining to Linux task intuition, shared process resources, per-thread stacks/TLS, and explicit object lifetime.

Prediction: the canonical basic and ownership modes report checked values; artifact metadata varies, but no output ordering is required.

Evidence: exact source ranges, binary identity, program assertions, and an ownership timeline from initialization through join and free.

Limits: no raw-clone programming, stack exhaustion, detached-thread race, TLS ABI disassembly, scheduling guarantee, or proof that one run covers every lifetime error.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Thread Model

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 11-threads-and-concurrency/challenges/day-001-thread-model.sh
```

- View lab source on GitHub

Day 2 — Data Races and the C Abstract Machine

Target time: approximately 3 hours

> A bad final count is evidence of a bug. A correct final count is not evidence that racy C was valid.

1. Observe the contrast safely

Run `challenges/day-002-data-races.sh`. It executes the checked mutex counter, then attempts a separately compiled ThreadSanitizer build of the intentional `race` mode. Predict that the mutex result is exactly 800000. Do not predict a particular racy total or promise that ThreadSanitizer runs.

The wrapper distinguishes three TSan outcomes: the compiler cannot build the instrumented program; the instrumented runtime cannot execute correctly in this environment; or a completed instrumented run reports the intentional race. Toolchain combinations can fail because of address-space layout, unsupported libc/runtime integration, container restrictions, or sanitizer runtime defects. An explicit skip is accurate evidence. It is not a pass for the program.

The ordinary race run is allowed to print any count, including the expected one. Because the program has undefined behavior, that number cannot validate the source. The bounded invocation is pedagogical evidence only.

2. Define a C data race

In the C abstract machine, a data race exists when different threads perform conflicting actions on the same memory location, at least one action is not atomic, and the actions are not ordered by happens-before. Two actions conflict when one modifies an object and the other reads or modifies the same object (with details for overlapping storage).

The key words are not “simultaneous in wall-clock time.” Happens-before is a language-level ordering assembled from sequenced-before relations within a thread and synchronizes-with relations supplied by operations such as mutex unlock/lock pairs, thread creation/joining, and suitable atomic release/acquire communication. If there is no required ordering, a lucky scheduler does not repair the source.

A data race gives undefined behavior. This is stronger than “one increment may be lost.” The implementation is no longer constrained to preserve the intuitive interleaving model for that execution. Debugging must therefore identify the missing ordering, not merely reproduce a numeric discrepancy.

3. Expand `counter++`

At source level, `counter++` is a read-modify-write expression, but an ordinary integer object does not make that operation atomic. A conceptual expansion is:

```
temporary = counter
temporary = temporary + 1
counter = temporary
```

Two workers can read the same old value and both store the same incremented value, apparently losing one update. Machine code may use a load, arithmetic instruction, and store, but the exact sequence depends on optimization and architecture. Even one machine instruction would not by itself establish C atomic semantics: alignment, compiler assumptions, ISA guarantees, and the C memory model all matter.

The canonical `unsafe_counter` is written by four threads without mutex or atomic operations. Those accesses conflict and lack happens-before ordering. The compiler may assume that a conforming program has no data race and transform code accordingly. Explaining only cache coherence or instruction interleaving misses this source-level fact.

4. Separate compiler ordering from CPU ordering

The compiler translates according to the C abstract machine. It may keep a value in a register, combine operations, eliminate loads, reorder independent work, or exploit impossible-under-defined-C assumptions. The CPU then executes instructions subject to its architecture's ordering and cache-coherence rules. Both layers affect observations, but they answer different questions.

`volatile` does not bridge the gap. It requests observable accesses useful for objects such as memory-mapped device registers or signal-related patterns within their proper rules; it does not make an increment indivisible and does not create inter-thread happens-before. Replacing the counter with `volatile` still leaves conflicting non-atomic accesses.

Compiler barriers, CPU fences, atomic operations, and mutexes are not interchangeable. A compiler barrier may restrict code motion without emitting a hardware instruction. An ISA fence orders specified machine-memory effects but does not retroactively make a non-atomic C race defined. C atomics provide language semantics that the implementation realizes with instructions, barriers, or library calls.

5. Read sanitizer evidence correctly

ThreadSanitizer instruments memory accesses and synchronization, maintaining a runtime model of happens-before. A useful report identifies two conflicting accesses, their thread creation sites, and associated stacks. That is much stronger diagnostic evidence than a bad counter total because it points to the protocol defect.

TSan is dynamic. It examines paths and schedules reached during that instrumented run. It can miss unexecuted races, unsupported synchronization, uninstrumented code, or races hidden by changed timing. Instrumentation also changes layout and scheduling. A clean report is not a proof of race freedom.

AddressSanitizer detects classes of invalid memory use; UndefinedBehavior Sanitizer checks selected operations; Valgrind tools use other instrumentation models. None should be described as a universal “C correctness checker.” Sanitizers may be mutually incompatible in one binary, and a sanitizer's own runtime availability must be tested, not inferred from successful compilation.

6. Repair according to the invariant

The mutex mode wraps every increment in lock/unlock. The protected invariant is that `mutex_counter` equals the number of completed critical-section updates. Unlock by one thread and subsequent lock by another establish synchronization, while mutual exclusion prevents overlapping updates.

The atomic mode uses `atomic_fetch_add_explicit(..., memory_order_relaxed)`. Relaxed is sufficient for the narrow invariant that no increments are lost: each operation is an indivisible read-modify-write in one modification order. It does not publish unrelated payload data. The final load occurs after all workers are joined, so thread completion supplies the needed phase boundary.

Choosing between a mutex and an atomic is not “slow versus fast.” A mutex can protect a multi-object invariant and compose a critical section. An atomic counter protects only its atomic operation unless a larger protocol explicitly connects other state through memory ordering.

7. Diagnose a race from source

For each shared object, list all accesses, mark writes, and identify the ordering edge between every potentially concurrent conflicting pair. A verbal claim such as “the producer goes first” is insufficient unless enforced by creation ordering, a mutex/condition protocol, atomic communication, or another documented primitive.

Check object lifetime too. A use-after-free can look like a race, and a race can permit a free to occur while another access is active. Ownership and happens-before are two separate obligations: the object must exist, and the access must be legal.

Do not fix symptoms with sleeps. Delays alter probability but establish no C synchronization. Do not fix a counter by narrowing the test until it usually passes. State the invariant, select a primitive that provides the required ordering and atomicity, and verify both normal and failure paths.

Practical challenge

Purpose: explain the intentional counter race using the C abstract machine, then contrast a real synchronization repair with dynamic diagnostic evidence.

Prediction: the mutex mode is exact; the race output is unconstrained; TSan either reports a race or is classified accurately as unavailable/inconclusive.

Evidence: source access inventory, happens-before graph, checked mutex result, compiler command, sanitizer diagnostic and exit status.

Limits: no exhaustive schedule exploration, portability proof from emitted instructions, claim that `volatile` synchronizes, or claim that a clean or unavailable TSan run proves race freedom.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Data Races

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 11-threads-and-concurrency/challenges/day-002-data-races.sh
```

- View lab source on GitHub

Day 3 — Mutexes, Ownership, Lock Ordering, and the Futex Boundary

Target time: approximately 3 hours

> A mutex is useful because it protects a named invariant, not because calls to `lock` and `unlock` happen to surround some code.

1. Check two protocols

Run `challenges/day-003-mutex-ownership.sh`. Predict an exact mutex count of 800000, one ownership transfer producing 42, and completion of the globally ordered two-lock mode. The wrapper also identifies source lines; your task is to explain why those lines define complete protocols.

In the counter mode, every read-modify-write of `mutex_counter` occurs while `counter_lock` is held. In the ownership mode, the parent allocates and initializes an object, passes its pointer at thread creation, refrains from accessing it while the worker may mutate it, joins, validates the returned pointer, and frees exactly once. These solve different problems: synchronization for a repeatedly shared object versus temporal transfer of exclusive access.

2. Name the protected invariant

Before adding a mutex, write the invariant in a sentence. For the counter: “while `counter_lock` is held, `mutex_counter` equals the number of completed protected increments.” For a linked list it might include reachability, link consistency, and size. For a bank transfer it likely spans two balances and a ledger entry.

Then list every location that reads or writes the protected fields. Locking only writers is insufficient when an unsynchronized reader conflicts with them. Using different mutexes for the same invariant is insufficient unless a documented partition proves the accesses concern disjoint state.

The mutex should be initialized before any user can reach it and destroyed only after no current or future operation can lock, unlock, or wait with it. Destroying an in-use mutex is a lifetime/protocol error. Static initialization works for the canonical process-lifetime mutexes; dynamic objects need explicit construction and teardown paths.

3. Understand ownership rules

A normal POSIX mutex has an owner while locked. The successful locker must unlock it; unlocking from another thread is not a portable handoff mechanism and can be undefined depending on mutex type. Error-checking mutexes can turn some mistakes into diagnostics, but they do not replace a correct protocol. Recursive mutexes can conceal accidental reentrancy and make invariants harder to audit.

Object ownership is broader than mutex ownership. “Owner” should answer who may mutate an object, who must release it, and how that responsibility moves. The canonical heap object is created by the parent, exclusively used by the worker between creation and join, and freed by the parent after join. No mutex is needed because there are no concurrent conflicting accesses.

A common failure is freeing task memory after calling `pthread_create`, on the assumption that the worker already copied it. Creation only starts a possible execution; it does not promise the worker has dereferenced the argument before the creator proceeds. Transfer must keep the allocation alive until an acknowledgment, join, queue protocol, or worker-owned free.

4. Derive happens-before from lock operations

Within one thread, initialization is sequenced before unlock. A mutex unlock synchronizes with a later successful lock of that mutex, so protected writes become visible to the next critical section. Mutual exclusion and ordering together let programmers reason in critical-section units instead of raw instruction interleavings.

The relation is tied to the same synchronization object and its valid lifetime. Unlocking mutex A does not automatically publish data to a thread that locks unrelated mutex B. Merely observing that a lock was uncontended does not remove its ordering semantics.

Keep critical sections large enough to preserve the invariant and small enough to avoid unnecessary contention. Splitting one logical update across two lock/unlock pairs can expose an invalid intermediate state. Conversely, performing blocking I/O or expensive callbacks while holding a widely used mutex can damage liveness and introduce unknown lock ordering.

5. Establish a global lock order

Multiple locks create a new obligation. If one path acquires A then B while another acquires B then A, both can hold one and wait forever for the other. The canonical ordered mode imposes lock 0 before lock 1 on every path. A rank system generalizes this: acquire in increasing rank, release normally in reverse, and never call code with unknown locking behavior while holding a ranked lock.

The policy must cover error paths and callbacks. “Usually A before B” does not prevent a rare deadlock. Sometimes try-lock, timed locking, lock coupling, or coarser locking is appropriate, but each needs an explicit recovery rule. A timeout detects lack of progress; it does not make partially completed work transactional.

Document whether a function requires a lock, acquires it internally, or must be called lock-free. Assertions or lock-order tooling can make the convention executable. Avoid exposing raw protected pointers beyond the lock's scope.

6. Locate the userspace/kernel boundary

On Linux, pthread mutex implementations commonly use an atomic userspace fast path when the mutex is uncontended. Contention may invoke `futex` operations so the kernel can block and wake tasks. This avoids a system call for every successful lock. Exact word layouts, adaptive spinning, robust-list behavior, and futex op selection are libc/version/architecture details.

“Mutexes are futexes” is too broad. POSIX specifies mutex behavior; libc may combine atomics, spinning, and Linux futex syscalls to implement it. The futex kernel interface coordinates waiting on a userspace value but does not own the high-level protected invariant. An uncontended trace may contain no futex syscall at all, while a contended run may still vary with timing.

If `strace -f -e futex` shows calls, that proves selected syscall-boundary events for that run. It does not reveal every lock acquisition, cache transfer, or scheduler decision. If it shows none, it does not prove mutex calls were optimized away; they may have remained on the fast path.

7. Handle failures without corrupting ownership

Pthread routines generally return an error number directly rather than setting `errno`. The canonical `check` helper prints the returned value and exits. Production code often needs structured unwinding: release locks already held, join created threads, free only still-owned objects, and preserve the first useful diagnostic.

Cancellation complicates cleanup because a thread can terminate at a cancellation point while owning resources. Cleanup handlers, restricted cancellation regions, or a no-cancellation policy around critical sections may be necessary. Robust mutexes add owner-death recovery semantics, but the next owner must repair state and mark consistency; they do not guarantee the interrupted invariant remained valid.

Debug lock problems by recording the invariant, owner, acquisition stack, rank, wait target, and lifetime. A debugger snapshot can show a current wait, but the source protocol explains whether it is legal.

Practical challenge

Purpose: connect mutex exclusion and synchronization to a named invariant, explicit heap ownership, global lock ordering, and Linux's optional futex slow path.

Prediction: protected count, ownership result, and ordered locking pass; any futex trace is environment- and schedule-dependent.

Evidence: checked program markers, source audit of every protected access, ownership timeline, lock-order graph, and narrowly scoped optional syscall trace.

Limits: no claim about mutex representation, fairness, uncontended syscall count, robust recovery, priority inversion policy, or all production failure paths.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check

- View quiz source on GitHub

Put it into practice — Mutex Ownership

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 11-threads-and-concurrency/challenges/day-003-mutex-ownership.sh
```

- View lab source on GitHub

Day 4 — Condition Variables, Predicate Loops, and Bounded Queues

Target time: approximately 3 hours

> A condition variable is a notification mechanism. The shared predicate, checked under its mutex, decides whether a thread may proceed.

1. Exercise the queue

Run `challenges/day-004-condition-queue.sh`. Predict 2000 consumed items, a sum of 2001000, an empty final queue, and the marker `condition_waits=while_predicate_loops`. Do not predict which producer or consumer runs first, how often either condition variable waits, or whether a spurious wakeup occurs.

The canonical queue has capacity eight, two producers, and two consumers. Small capacity forces coordination without requiring stress. Producers insert positive values; after both producers join, the parent inserts one `-1` sentinel per consumer. Every consumer returns a heap-allocated partial result, which the parent joins, combines, and frees.

The final count and arithmetic sum jointly detect many missing, duplicate, or corrupted-item failures. They do not prove FIFO order per producer or exercise every schedule.

2. State predicates before operations

The queue's state is `items`, `head`, `tail`, and `count`, all protected by `queue.lock`. Two predicates control progress:

```
can_put := count < CAPACITY
can_get := count > 0
```

`put` locks, waits while `can_put` is false, writes at `tail`, advances the index modulo capacity, increments `count`, signals `not_empty`, and unlocks. `get` locks, waits while `can_get` is false, reads at `head`, advances, decrements, signals `not_full`, and unlocks.

The structural invariant is `0 <= count <= CAPACITY`, with head and tail always valid indices. Mutex exclusion keeps each transition atomic relative to other queue operations. Condition variables avoid busy-waiting when no legal transition is currently available.

3. Understand the atomic wait transition

`pthread_cond_wait(&condition, &mutex)` requires the caller to hold the mutex. Conceptually it releases the mutex and blocks as one coordinated operation, then reacquires the mutex before returning. That coordination prevents the classic lost-wakeup gap where a waiter checks a false predicate, another thread makes it true and signals, and only then the waiter starts sleeping.

The mutex—not the condition variable—protects the predicate. Every code path that changes `count` does so under `queue.lock`. The waiter checks the predicate under that same lock. The signal occurs while the state transition is still protected in this implementation, and the awakened waiter cannot inspect the new state until it reacquires the mutex.

A signal with no current waiter is not stored as a future permit. Condition variables are not counting semaphores. The queue state persists; notifications only prompt waiters to check that state again.

4. Always use a predicate loop

The wait must be inside `while`, not `if`. POSIX permits spurious wakeups, so a wait can return without the program-specific predicate becoming true. Even without a spurious wakeup, another awakened or already-running thread may acquire the mutex first and consume the only item or slot. By the time this thread reacquires the lock, its predicate can be false again.

The correct pattern is:

```
lock
while predicate is false:
    wait(condition, lock)
perform transition
possibly notify
unlock
```

This pattern makes wakeup causes irrelevant to correctness. A wakeup is an invitation to re-evaluate state. Code that attempts to distinguish a “real” signal from a spurious one usually models the condition variable incorrectly.

Timeouts add another possible return, but timeout and predicate transition can race. After a timed wait, re-check the predicate under the mutex and interpret the API's return carefully. Use the clock configured for the condition variable, and expect wall-clock changes if it uses realtime.

5. Choose signal or broadcast from the transition

`pthread_cond_signal` wakes at least one eligible waiter; `broadcast` wakes all. In this queue, one insertion creates at least one unit of consumer progress, and one removal creates one unit of producer progress, so signaling one is a reasonable policy. Correctness still depends on the predicate loop.

Broadcast is useful when one transition may enable many waiters or when different predicates share a condition variable and all must re-check. It can also create a thundering herd: many threads wake, contend for the mutex, and most return to sleep. Signal can be insufficient in protocols where multiple units become available or where the selected waiter cannot make the progress another waiter could.

Notification placement is a protocol choice. Updating the predicate must happen under the mutex. Signaling while holding it often makes the relationship obvious; signaling after unlock can be correct for carefully designed protocols but requires proof that lifetime and predicate transitions cannot race with waiter setup or object destruction.

6. Prove termination and ownership

After producers join, no more positive values will be created. The parent then enqueues exactly one sentinel per consumer. Queue FIFO behavior ensures all previously queued work precedes subsequently inserted sentinels. Each consumer stops after one sentinel, so every consumer can terminate without a shared “done” race.

This shutdown technique assumes `-1` is outside the valid data domain. In a general queue, use a tagged item, a protected closed flag, reference counts, or an explicit close operation. If producers can fail, the parent must still decide how to close and wake blocked consumers. A missing sentinel can leave a join blocked forever; too many can terminate consumers before all work under a different ordering protocol.

Each consumer allocates its result and returns ownership through `join`. Allocation failure returns `NULL`; the parent detects it. One limitation is that the other threads have already completed by then, so this demo reports failure rather than implementing coordinated early cancellation.

7. Debug queue hangs systematically

For a hang, inspect all threads and classify each: running, waiting for the mutex, waiting on `not_empty`, waiting on `not_full`, or blocked in join. Record `count`, `head`, `tail`, producer completion, number of sentinels inserted, and the owner of the mutex. Do not mutate shared variables in the debugger before capturing evidence.

For corruption, verify every state access holds the intended mutex. Then check index arithmetic, item domain, duplicate ownership, consumer result lifetime, and shutdown transitions. Sanitizers can detect invalid memory but do not prove the queue protocol. Helgrind/DRD or TSan may help where available, with their respective runtime and modeling limits.

A single successful queue run establishes the checked count, sum, and final state for that artifact and execution. It does not establish fairness, maximum throughput, absence of starvation, or correctness for multiple producers that terminate unexpectedly.

Practical challenge

Purpose: derive a bounded producer-consumer queue from protected predicates, atomic wait transitions, re-check loops, and explicit shutdown ownership.

Prediction: all 2000 values are consumed exactly in aggregate, the expected sum is observed, and queue count returns to zero.

Evidence: checked count/sum, source audit of every queue-state access, predicate transition table, and an ownership account for sentinels and result allocations.

Limits: no guaranteed wake order, fairness, observed spurious wakeup, failure-injection recovery, cancellation safety, throughput conclusion, or proof over every schedule.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Condition Queue

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 11-threads-and-concurrency/challenges/day-004-condition-queue.sh
```

- View lab source on GitHub

Day 5 — C11 Atomics, Memory Orders, and Architecture Mappings

Target time: approximately 3 hours

> The portable claim comes from the C memory model. Assembly and hardware observations explain an implementation; they do not redefine the language.

1. Run two deliberately different atomic experiments

Run `challenges/day-005-atomics-memory-order.sh`. Predict an exact atomic counter of 800000 because each relaxed fetch-add participates in the atomic object's modification order. Predict only that the litmus completes 100000 rounds and prints a nonnegative `both_zero` count. Do not predict that both-zero must or must not appear on this host.

The wrapper may also request compiler assembly for the canonical source. Treat it as evidence for that compiler, flags, source, and target only. Optimizers can transform surrounding code, and atomics that look similar in C can map differently depending on width, alignment, target extensions, or whether an operation is lock-free.

Most importantly, neither seeing nor failing to see an outcome on x86-64 proves portable C behavior. The litmus source itself uses relaxed atomics and is defined; the C model determines its allowed executions.

2. Begin with atomicity and modification order

An atomic object is accessed through atomic operations. Each atomic object has a single modification order for its writes and read-modify-write operations. `atomic_fetch_add` is indivisible with respect to competing atomic operations on that object, so four workers cannot lose increments as the non-atomic mode can.

`memory_order_relaxed` supplies atomicity and modification-order coherence but no release/acquire synchronization for unrelated ordinary objects. It is ideal for independent statistics where only the count matters. It is not sufficient to publish a pointer plus initialized payload merely because the pointer store is atomic.

After joining all workers, the parent loads the counter. Join orders completed thread actions before the joiner proceeds. The demo's exact result therefore comes from atomic increments plus the completion phase, not from relaxed ordering magically synchronizing every object.

3. Build a release/acquire publication

Suppose a producer fills ordinary payload fields, then performs a release store of `ready = true`. A consumer performs an acquire load that reads that store (or its release sequence) and then reads the payload. The release synchronizes with the acquire, creating happens-before from payload initialization to payload consumption.

The acquire must actually observe the appropriate release relationship. An acquire load returning `false` does not import the producer's writes. A relaxed load of `ready` preserves atomicity but does not establish this publication edge. The protocol normally loops or otherwise waits until the expected value is acquired.

Release prevents prior relevant actions from moving after publication in the abstract semantics; acquire prevents subsequent relevant actions from moving before consumption. They are directional. Using release on a load or acquire on a plain store is not a meaningful substitute, and the API restricts which orders each operation accepts.

4. Place stronger orders carefully

`memory_order_seq_cst` adds all sequentially consistent operations to one global order consistent with required per-thread order. It simplifies many arguments but does not turn a collection of operations into one transaction. Ordinary data still needs a valid synchronization protocol, and invariants spanning several atomics can expose intermediate states.

Acquire-release read-modify-write operations combine both directions. `memory_order_acq_rel` can consume prior publication and publish preceding updates while atomically changing one object. A failed compare-exchange does not perform the write, so it has a separately specified failure order that cannot include release semantics.

`memory_order_consume` has historically been difficult to implement as its dependency-based intent suggested; portable teaching and production guidance commonly uses acquire instead. Always check the language/toolchain version rather than assuming an informal dependency rule.

Fences can connect carefully structured atomic operations, but they are easier to misuse than release/acquire on the communicating object. A fence does not make nearby non-atomic races legal by proximity.

5. Analyze the store-buffering litmus

The canonical litmus has atomic `x` and `y`, initially zero each round. Thread 0 stores 1 to `x` relaxed and loads `y` relaxed. Thread 1 stores 1 to `y` relaxed and loads `x` relaxed. Barriers separate rounds but do not impose a cross-thread order between the two atomic operations inside a round.

Both loads may read zero under relaxed C semantics. Each atomic still has a coherent modification order; there is simply no release/acquire edge requiring either load to observe the other thread's store. The barrier after the operations lets the controller safely inspect the result array, whose slots are written by separate workers and not reused until phase completion.

Changing operations to sequential consistency alters the allowed global-order argument: both-zero cannot be placed consistently with each store preceding its following load and both loads preceding the other stores. Validate such claims against the exact source; compiler transformations and barrier semantics matter.

6. Map cautiously to x86-64, AArch64, and RISC-V

On x86-64's commonly described TSO model, ordinary aligned loads/stores already provide relatively strong ordering, but store-to-load relaxation through store buffers matters to the litmus. A compiler may emit plain instructions for relaxed and release/acquire operations and use locked operations or fences where stronger guarantees require them. Exact lowering depends on the operation and compiler.

AArch64 has a weaker ordering model and dedicated acquire/release forms such as `LDAR` and `STLR`; relaxed operations can use ordinary `LDR`/`STR`, while read-modify-write may use exclusive loops or LSE atomics. Barriers such as `DMB` can implement stronger order. Architectural extensions and target flags change the sequence.

RISC-V's base weak memory model uses ordering bits on atomic instructions and fences; implementations and compiler mappings depend on the A extension, profiles, and evolving architecture/toolchain support. Loads/stores plus `fence` or acquire/release annotations on AMOs may realize C orders.

These are mapping landmarks, not a lookup table. For each claim record compiler version, target triple, optimization, ISA extensions, ABI, source, and emitted instructions. Never transfer an observed x86 result into a portable guarantee.

7. Separate compiler and hardware evidence

Compiler explorers and `-S` output show selected lowering. They cannot show runtime cache state or all legal compiler choices. ISA manuals define machine ordering, not C object lifetime or data-race rules. Hardware counters and litmus tools measure or explore architecture behavior, but a user-space test has finite samples and environmental noise.

Start a correctness proof in C: objects, operations, modification orders, sequenced-before, synchronizes-with, and happens-before. Then inspect compiler output to understand costs or diagnose a suspected lowering issue. Finally use architecture documentation and measurements to explain the machine behavior. Do not reverse that hierarchy.

Atomics also do not solve reclamation automatically. A lock-free pointer can be atomically updated while another thread dereferences freed storage. Hazard pointers, epochs, reference counting, or another lifetime scheme is required. Lock-free means some operation makes system-wide progress; it does not mean wait-free, fair, simple, or faster.

Practical challenge

Purpose: distinguish atomicity from ordering, derive release/acquire happens-before, interpret a relaxed litmus, and compare representative ISA mappings without weakening the portable argument.

Prediction: the relaxed counter is exact; litmus count varies; assembly depends on compiler and host target.

Evidence: C source protocol, checked counters, finite litmus observations, compiler identity/options, emitted assembly, and cited ISA rules.

Limits: no portability claim from x86 observation, no exhaustive hardware outcome search, no promise of lock freedom, no universal instruction mapping, and no automatic solution to multi-object invariants or reclamation.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Atomics Memory Order

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 11-threads-and-concurrency/challenges/day-005-atomics-memory-order.sh
```

- View lab source on GitHub

Day 6 — Deadlock, Starvation, False Sharing, and Scalability

Target time: approximately 3 hours

> Correct final values establish safety properties. They do not establish that every thread progresses promptly or that added cores improve throughput.

1. Run bounded liveness and timing experiments

Run `challenges/day-006-liveness-scalability.sh`. Predict that the detector reports a lock cycle without hanging, the ordered mode completes, and both false-sharing timings are nonnegative. Do not predict which timed lock returns `ETIMEDOUT`, a fixed adjacent/padded ratio, or any speedup.

The deadlock detector deliberately synchronizes two workers after each has acquired a different mutex. Each then attempts the other mutex with a deadline. At least one attempt must time out, after which locks are released and threads join. This reproduces the wait-for cycle while bounding the test.

The false-sharing mode places two atomic counters first adjacent and then in separately aligned/padded structures. It records elapsed time for one run of each. The program itself labels the timing as noisy and not a correctness test.

2. Distinguish safety from liveness

Safety means “nothing bad happens”: no data race, no broken invariant, no double free. Liveness means “something good eventually happens”: an operation completes, a waiter is served, the system continues to make progress. Mutexes can establish safety while a bad acquisition graph destroys liveness.

A deadlock is a state in which participants wait in a cycle that cannot resolve without external intervention. The classic necessary conditions are mutual exclusion, hold-and-wait, no forced preemption of resources, and circular wait. Breaking any one can prevent this form of deadlock. Global lock ordering directly prevents circular wait.

Not every hang is deadlock. A thread can wait for a condition that no producer will ever make true, block on I/O, spin on a stale protocol, miss shutdown, or wait for a thread that failed before signaling. Build a wait-for graph from actual owners and dependencies before labeling the symptom.

3. Treat timeouts as detection, not repair

`pthread_mutex_timedlock` bounds one wait. A timeout says the mutex was not acquired by the deadline; in the controlled experiment, synchronized opposite ownership makes that evidence of the intended cycle. In general, a timeout can also result from preemption, overload, priority inversion, or an overly short deadline.

After timeout, software must restore or abandon partial state coherently. The canonical workers have not changed shared application data while holding only their first lock, so they can release it. A transaction that updates one object before timing out on another needs rollback, idempotent retry, versioning, or a different design. “Retry forever” can turn deadlock avoidance into livelock.

Use a monotonic clock where the API allows it for duration-based deadlines. The canonical mutex timed lock uses `CLOCK_REALTIME` because that is the portable default for this API form; realtime adjustment is therefore a stated measurement limitation.

4. Recognize starvation and fairness limits

Starvation occurs when one thread repeatedly fails to make progress while others do. There need not be a cycle. An unfair lock, priority policy, constant arrival of competing work, or reader-preference design can starve a participant while aggregate throughput remains high.

POSIX mutexes generally should not be assumed to provide FIFO admission unless a specific protocol says so. A successful thousand-iteration test cannot prove starvation freedom. Measure per-thread wait distributions and maximum waits, not only total throughput, and identify scheduler policy, CPU placement, and load.

Priority inversion occurs when a high-priority thread waits for a lock held by a low-priority thread while medium-priority work delays that owner. Priority inheritance mutex protocols may help in real-time designs, but availability, permissions, scheduler configuration, and transitive cases matter. Ordinary desktop timing does not validate a real-time bound.

5. Explain false sharing

Coherent caches typically move ownership at cache-line granularity. Two threads updating distinct atomic objects on the same line do not have a C data race, yet each write can invalidate the other's cached copy. The line “ping-pongs,” adding coherence traffic and latency. This is false sharing: independent language objects interfere because of physical layout.

Padding or alignment can place hot writable fields on different lines. The canonical `_Alignas(64)` and padding are an experiment, not a universal cache line declaration. The atomic type size, structure layout, allocator placement, compiler, CPU line size, and adjacent objects must be checked. Over-padding can waste cache and memory bandwidth.

False sharing differs from true sharing, where threads intentionally update the same synchronization object or data. It also differs from a data race. Relaxed atomics make each benchmark update defined while preserving the coherence cost; they do not eliminate contention.

6. Measure scalability responsibly

Speedup is `T1 / Tp`; efficiency is speedup divided by worker count. Amdahl's law reminds us that serial work, synchronization, and overhead cap speedup. Contention can make additional threads slower. Work partitioning, NUMA placement, memory bandwidth, allocator behavior, and operating-system noise all influence results.

A useful benchmark includes warmup where justified, repeated samples, distribution summaries, controlled input, correctness checks, and environment metadata. Pinning threads can reduce migration but also create an artificial placement; report it. CPU frequency scaling, simultaneous multithreading, background load, thermal limits, and virtualization must be considered.

The canonical benchmark is deliberately too small for a scalability claim. It runs adjacent first and padded second, so order effects are confounded. It does not pin CPUs, inspect line addresses, repeat trials, or report hardware counters. Its value is generating a hypothesis and teaching measurement limits.

7. Improve designs before micro-optimizing

Reduce shared writes through per-thread counters combined after join, sharded locks, batching, immutable snapshots, or ownership transfer. Such changes can remove contention and false sharing together. They also change memory usage, latency, complexity, and consistency semantics.

Lock-free algorithms can improve some workloads but introduce atomic hot spots, retry storms, ABA concerns, and difficult reclamation. A lock-free progress guarantee is system-wide, not necessarily fair to each thread. Wait-free algorithms provide a bounded per-operation progress guarantee but are harder and may have larger constants.

Optimize only after preserving an executable correctness oracle. Compare throughput and tail latency at multiple thread counts against a single-thread baseline. If results differ, use profilers and hardware counters where available, labeling sampled evidence and event semantics. Avoid attributing a timing ratio to false sharing without confirming layout and coherence events.

Practical challenge

Purpose: observe a bounded wait-for cycle, verify global ordering, measure a false-sharing hypothesis, and separate correctness, liveness, and performance claims.

Prediction: timed detection and ordered locking terminate; adjacent and padded runs produce variable timings with no required winner.

Evidence: lock graph, timeout return codes, completion markers, binary and environment identity, object addresses/layout if collected, and repeated timing distributions for any performance report.

Limits: one run does not prove fairness, starvation freedom, cache-line causality, realtime bounds, or scalability; the benchmark lacks affinity, counter evidence, randomized order, and statistical power.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Liveness Scalability

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 11-threads-and-concurrency/challenges/day-006-liveness-scalability.sh
```

- View lab source on GitHub

Day 7 — GDB, Valgrind, Sanitizers, and System Comparison

Target time: approximately 3 hours

> A concurrency debugging report must identify the artifact, preserve the failure, connect evidence to a protocol, and state what each tool cannot see.

1. Generate the evidence bundle

Run `challenges/day-007-debug-report.sh`. It creates a fresh temporary report, runs the canonical deterministic Week 11 suite, captures identity, executes intentional bounded failures, and tries optional GDB, Valgrind, AddressSanitizer/UBSan, and ThreadSanitizer checks.

Predict `WEEK11 PASS`. Predict that the deadlock detector terminates and that the data-race mode remains outside deterministic correctness testing. Do not predict optional-tool availability. A tool binary can exist while its runtime is unusable; this is especially relevant for TSan.

Keep the full diagnostics when a tool cannot run. “SKIP: TSan runtime unavailable or no report” is more accurate than claiming the race passed. Likewise, a debugger blocked by ptrace policy has produced no stopped-state evidence.

2. Establish reproducible identity

Record UTC time, `uname -a`, architecture, compiler and libc versions, exact canonical project path, source and binary hashes, build flags, optimization, and invocation. For optional tools record their versions and exit statuses. A report against a different binary cannot explain the failure.

Concurrency failures are schedule-sensitive. Preserve standard output and error, iteration counts, environment variables, CPU count/placement if known, and whether instrumentation was active. Avoid “fixing” the reproduction with logging before saving the original evidence; I/O can perturb timing.

If source changes, make a new identity section. Line-level debugger and sanitizer stacks are meaningful only against matching debug information and binaries.

3. Use GDB for state, not proof of absence

In GDB, `info threads`, `thread apply all bt`, mutex/condition state where debug symbols permit, and selected shared variables can capture a stopped snapshot. For a hang, classify every thread's current wait and construct a wait-for graph. For lifetime errors, inspect the allocation owner and frames that could still hold pointers.

Stopping changes scheduling. A snapshot can establish that thread A currently waits in a lock path while B owns or waits elsewhere; it cannot by itself prove the complete historical cause. Mutex internals are libc-private and version dependent, so do not hard-code field interpretations without matching sources.

Breakpoints serialize execution and may hide races. Watchpoints are limited hardware resources and observe selected accesses. GDB's scheduler-locking controls can help isolate a transition but create an execution unlike normal operation. Record every debugger intervention.

4. Use Valgrind and sanitizers according to their models

Valgrind Memcheck can detect invalid reads/writes, use of uninitialized values, and leaks in executed code, though it is slow and can significantly alter thread timing. Valgrind's thread tools, where installed, have separate happens-before models and options; “Valgrind clean” without naming the tool is ambiguous.

AddressSanitizer uses compiler instrumentation and shadow memory for many spatial and temporal errors. LeakSanitizer integration varies. UBSan detects selected undefined operations but is not generally a data-race detector. ThreadSanitizer instruments memory and synchronization accesses to report dynamic races.

For TSan distinguish:

1. compiler rejects the instrumentation;
2. binary builds but runtime fails before meaningful execution;
3. run completes without reporting the targeted race;
4. report identifies conflicting accesses and exits by configured status.

Only the fourth is positive detection evidence. The third is inconclusive, not a correctness result. Any dynamic tool can miss unexecuted paths and can perturb the schedule.

5. Preserve intentional failures

The canonical `thread_lab race` is intentionally undefined and must never be folded into a suite that treats exit zero or a numeric total as correctness. Its purpose is to feed a race detector and illustrate why output cannot validate racy C.

The `deadlock_lab detect` mode intentionally creates opposite acquisition order, but timed lock bounds the failure. It checks that at least one attempt times out and then releases both first locks. This is an expected detector, not permission to deploy timeout-based partial transactions.

False-sharing and relaxed-litmus modes are intentional non-assertions. Their variable values are evidence to classify, not thresholds to force. A report that marks a particular ratio or `both_zero` count as required has converted an observation exercise into a flaky test.

6. Debug from invariant to root cause

For a wrong result, list the involved objects and every access. For each conflicting pair, locate its happens-before edge. For a crash, verify object lifetime first: allocation, publication, owners, final access, destruction. For a hang, build a wait-for graph and inspect shutdown predicates.

Minimize while preserving the defect. Keep the same optimization and instrumentation status during comparison. Then add one tool at a time: sanitizer for dynamic access evidence, debugger for a stopped snapshot, syscall trace for futex/blocking boundaries, compiler output for lowering, and profiler/counters for performance hypotheses.

End every conclusion with scope. “TSan reported concurrent writes to `unsafe_counter` in this instrumented run” is defensible. “The CPU reordered the increment” is not established by that report. “Padded was faster once” does not establish false sharing as the cause.

7. Compare ToyOS, xv6, and Linux carefully

Pin exact revisions before comparison. “ToyOS” is course-specific; inspect whether its current milestone even has multiple execution contexts, shared address spaces, blocking locks, or a scheduler. Absence is a valid result, not a defect. Its simplest lock may disable interrupts on one CPU, which does not automatically generalize to SMP.

xv6 variants differ. The common teaching kernels have a compact process table, scheduler, sleep/wakeup mechanism, spinlocks, and often per-process address spaces; lab branches may add kernel threads, copy-on-write, or other features. Read the chosen RISC-V revision's `proc.c`, lock implementation, sleep/wakeup code, and memory subsystem. Do not attribute a lab extension to base xv6.

Linux presents pthreads through libc over kernel tasks, supports resource sharing through clone-family interfaces, futex-backed blocking primitives, rich scheduling, TLS ABI support, sanitizers in userspace toolchains, ptrace, procfs, and performance tooling. Complexity expands possible evidence and lifetime obligations; it does not make teaching kernels “wrong.”

Use a question-based comparison:

- What is the schedulable entity?
- Can execution contexts share one user address space?
- How is a blocked waiter represented and awakened?
- Which primitive supplies mutual exclusion on one CPU and on SMP?
- Where are stack and TLS state established?
- What establishes lifetime after exit/join?
- Which observations come from symbols, serial output, debugger, procfs, tracing, or instrumentation?

Separate interface guarantees from implementation landmarks in every system.

8. Write the final report

Include identity, reproduction, ownership/lifetime, happens-before graph, counter race, mutex invariant, queue predicates, atomic ordering, liveness graph, performance caveats, tool results, intentional failures, and three-system comparison. For every tool, name the captured artifact and diagnostic, then state its blind spots.

A strong causal conclusion looks like: “Two instrumented worker stacks report conflicting non-atomic writes to `unsafe_counter`; source inspection finds no synchronizes-with edge between increments. The C execution therefore has a data race and undefined behavior. The report covers this instrumented run and does not prove TSan observes every schedule.”

Conclude with cleanup and non-claims. All deterministic modes completed, the deadlock exercise was bounded, every created thread was joined, returned heap objects were freed, and temporary report paths were printed. No x86 observation was used as portability proof.

Practical challenge

Purpose: produce a reproducible concurrency debugging report joining source protocols, runtime assertions, debugger/tool evidence, intentional failures, and revision-aware system comparison.

Prediction: deterministic Week 11 tests pass; optional tools either produce scoped evidence or an explicit diagnostic/skip; intentional race output is never accepted as correctness.

Evidence: versions, hashes, commands, exit statuses, complete logs, stack traces, happens-before and wait-for graphs, source landmarks, and limits.

Limits: no exhaustive schedule proof, guaranteed ptrace/sanitizer runtime, kernel rebuild, stress campaign, fairness proof, portable claim from one ISA, or comparison without pinned ToyOS/xv6/Linux revisions.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Debug Report

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 11-threads-and-concurrency/challenges/day-007-debug-report.sh
```

- View lab source on GitHub